AVImark COM Server — Complete Technical Analysis

AVImarkCOMServer.dll — 12 COM Interfaces, Full API Reference

Rosslyn Veterinary Clinic • ROS-SVR (Windows Server 2019) • Generated April 2, 2026 • CONFIDENTIAL

Table of Contents

1. Executive Summary
2. DLL Technical Profile
3. COM Architecture Overview
4. Interface Reference (12 Interfaces, Full Methods)
5. API Method Catalog — Detailed
6. Security Analysis
7. Integration Opportunities
8. Automation Use Cases
9. Registration Status & Next Steps
10. Appendix: Raw Data

1. Executive Summary

The AVImark COM Server (`AVImarkCOMServer.dll`) is a 32-bit in-process COM automation server built by Covetrus Inc. (formerly Henry Schein) as part of AVImark Classic v2022.10.6. It exposes **12 dispatch interfaces** providing programmatic access to virtually every function in the AVImark practice management system.

12	74.7 MB	65+
COM INTERFACES	DLL SIZE	API METHODS

7	3	10
DLL EXPORTS	EMBEDDED TYPELIBS	PE SECTIONS

Key Finding: This COM server provides full **read AND write** access to AVImark data — including creating invoices, posting history, creating purchase orders, adding items, modifying field values, and scheduling appointments. Combined with the existing ETL pipeline (30 tables, 2.58M rows in Supabase), this unlocks bidirectional integration.

Registration Status: The DLL includes `DllRegisterServer` but registration via `regsvr32` silently fails on ROS-SVR. Likely requires AVImark workstation context or specific runtime dependencies. Further investigation needed.

2. DLL Technical Profile

File Name	`AVImarkCOMServer.dll`
Full Path	`C:\AVImark\AVImarkCOMServer.dll`
File Size	78,285,040 bytes (74.7 MB)
Architecture	32-bit (x86) — PE32
Compiler	Embarcadero Delphi (Borland Object Pascal)
File Version	2022.10.6.0
Product Version	2022.10.6.0
Product Name	AVImark
Company	Covetrus Inc.
Copyright	Copyright 2022 Covetrus Inc.
Description	AVImark COM Server Library
Internal Name	`22.10.6: f10f72680df7baeef2f6ffe6af7903256da1c82c (RELEASE;AVImarkClient;)`

Created	December 8, 2021
Last Modified	April 5, 2023
COM Type	In-Process Server (InprocServer32)
Threading Model	Apartment (STA) — typical for Delphi COM

DLL Exports (7)

#	EXPORT NAME	PURPOSE
0	`DllCanUnloadNow`	Standard COM — checks if DLL can be unloaded
1	`DllGetClassObject`	Standard COM — returns class factory for CLSID
2	`DllRegisterServer`	Standard COM — self-registration (writes to registry)
3	`DllUnregisterServer`	Standard COM — removes registration
4	`TMethodImplementationIntercept`	Delphi RTTI method interception
5	`__dbk_fcall_wrapper`	Delphi debugger function call wrapper
6	`dbkFCallWrapperAddr`	Delphi debugger address pointer

PE Section Map

SECTION	VIRTUAL ADDRESS	RAW OFFSET	RAW SIZE	PURPOSE
`.text`	0x1000	0x400	59,722,752	Executable code (56.9 MB)
`.itext`	0x38F6000	0x38F5000	250,368	Initialization code
`.data`	0x3934000	0x3932200	996,864	Initialized data
`.bss`	0x3A28000	—	0	Uninitialized data
`.idata`	0x3AFC000	0x3A25800	31,744	Import directory
`.didata`	0x3B04000	0x3A2D400	28,672	Delayed import data
`.edata`	0x3B0B000	0x3A34400	512	Export directory
`.rdata`	0x3B0C000	0x3A34600	512	Read-only data
`.reloc`	0x3B0D000	0x3A34800	5,501,952	Relocations (5.2 MB)
`.rsrc`	0x404D000	0x3F73C00	11,719,680	Resources (11.2 MB — includes 3 TypeLibs, icons, images)

Embedded Type Libraries (3)

#	FILE OFFSET	CONTENT
1	60,403,532 (0x399AF4C)	ICC Color Profile (sRGB) — embedded image resource
2	67,421,620 (0x404A734)	**AVImark COM Server Type Library** — all 12 interfaces
3	77,012,419 (0x4970DC3)	ICC Color Profile (Adobe RGB 1998) — embedded image resource

3. COM Architecture Overview

The AVImark COM Server follows the standard Microsoft COM (Component Object Model) architecture. It is an **in-process server** (DLL-based) that runs within the calling process's address space. Each interface is an IDispatch automation interface, meaning it supports late-binding from scripting languages (VBScript, PowerShell, Python via win32com).

Interface Hierarchy

```
AVImarkCOMServer.dll (InprocServer32, Apartment-threaded)
│
├── IAVIClientInProc          — Client/owner lookup, search, invoice creation
│   └── ProgID: AVIClientInProc
│
├── IAVIPatientInProc         — Patient/animal records, reminders, charts
│   └── ProgID: AVIPatientInProc
│
├── IAVITreatmentInProc       — Treatment code definitions, models
│   └── ProgID: AVITreatmentInProc
│
├── IAVIInventoryInProc       — Inventory tracking, usage history
│   └── ProgID: AVIInventoryInProc
│
├── IAVIHistoryInProc         — Medical/clinical history, lab results
│   └── ProgID: AVIHistoryInProc
│
├── IAVIEntryInProc           — System table entries, code lookups
│   └── ProgID: AVIEntryInProc
│
├── IAVISystemTableInProc     — Lookup table access (species, breeds, etc.)
│   └── ProgID: AVISystemTableInProc
│
├── IAVIOrderInProc           — Purchase order creation, item management
│   └── ProgID: AVIOrderInProc
│
├── IAVIFileInProc            — Raw V2$ file I/O with authentication
│   └── ProgID: AVIFileInProc
│
├── IAVIEstimateInProc        — Estimates/quotes management
│   └── ProgID: AVIEstimateInProc
│
├── IAVIAppointmentInProc     — Scheduling, doctor/room calendars
│   └── ProgID: AVIAppointmentInProc
│
└── IAVICensusInProc          — Whiteboard/census, in-clinic status
    └── ProgID: AVICensusInProc
```

Access Model

> **Authentication:** The `IAVIFileInProc` interface includes a `Login(Password)` method, indicating the COM server requires authentication before allowing data access. The password is likely the same as the AVImark server password found in `AVImarkServer.ini` (stored as a hex-encoded hash).

Communication Pattern

All interfaces use a **handle-based navigation pattern**:

1. **Search** — `GetFirstByName(key)` returns a handle (record pointer)
2. **Navigate** — `GetNextByName(key)` iterates through results
3. **Read** — `GetMoreStuff(handle)` returns `FieldNames` + `FieldValues`
4. **Write** — `SetFieldValues()` or `SetFieldValuesByName()` modifies data
5. **Create** — `CreateInvoice()`, `CreateOrder()`, `AddNewEntry()`, `AddRecordEx()`

4. Interface Reference — 12 Interfaces

🧑 IAVIClientInProc READ/WRITE

Purpose: Client (pet owner) lookup, search, patient/account linking, and invoice creation.

ProgID: `AVIClientInProc`

METHOD	PARAMETERS	ACCESS	DESCRIPTION
`GetFirstByName`	`NameKey → Value`	READ	Find first client by last name. Returns handle/value.
`GetNextByName`	`NameKey`	READ	Iterate to next client matching name.
`GetFirstByNameKey`	`NameKey`	READ	Search by full name key (last,first).
`GetNextByNameKey`		READ	Next result for name key search.
`GetFirstByPhone`	`PhoneKey`	READ	Find client by phone number.
`GetNextByPhone`		READ	Next result for phone search.
`GetFirstByPhoneKey`	`PhoneKey`	READ	Phone key-based search.
`GetFirstByWholePhoneKey`	`PhoneKey`	READ	Full phone number match.
`GetNextByPhoneKey`		READ	Next phone key result.
`GetNextByWholePhoneKey`		READ	Next full phone result.
`GetClientsByEmailOrPhone`	`Email, Phone`	READ	Multi-field client search.
`GetPatientHandles`	`ClientHandle, IncludeInactive`	READ	Get all animals for a client. Boolean flag for inactive.
`GetAccountingHandles`		READ	Get financial transaction handles for client.
`GetByFolder`	`Folder`	READ	Find client by folder number.
`GetFirstByMoreStuff`	`MoreStuffKey`	READ	Extended field search.
`GetNextByMoreStuff`		READ	Next extended search result.
`GetFirstByMoreStuffKey`	`MoreStuffKey`	READ	Key-based extended search.
`GetNextByMoreStuffKey`		READ	Next key-based result.
`GetMoreStuff`	`RecordHandle → FieldNames, FieldValues`	READ	Get all fields for a record. Returns parallel arrays of names and values.
`CreateInvoice`	`Date`	WRITE	Create a new invoice for the current client context. ⚡

🐾 IAVIPatientInProc READ

Purpose: Patient (animal) record access, reminders, history, chart printing.

ProgID: `AVIPatientInProc`

METHOD	PARAMETERS	DESCRIPTION
`GetByCode`	`Code`	Look up patient by code.
`GetByTagNo`	`TagNo, Microchip`	Find animal by tag number or microchip ID.
`GetReminderHandles`	`PatientHandle`	Get all due/overdue reminders for a patient.
`GetHistoryHandles`		Get medical history handles for current patient.
`GetAppointmentHandles`		Get appointment handles for current patient.
`GetViewHandles`	`ViewName`	Get handles for a named view/list.
`PrintPatientChart`	`PatientRecord`	Trigger chart print for a patient.
`GetModelHandles`	`TreatmentHandle`	Get model/template handles for treatment.
`Values`		Get current record values.

💊 IAVITreatmentInProc READ

Purpose: Treatment/procedure code lookups and model access.

ProgID: `AVITreatmentInProc`

METHOD	PARAMETERS	DESCRIPTION
`GetModelHandles`	`TreatmentHandle`	Get treatment models/templates by handle.

📦 IAVIInventoryInProc READ

Purpose: Inventory item tracking and usage history.

ProgID: `AVIInventoryInProc`

METHOD	PARAMETERS	DESCRIPTION
`GetInventoryUsedHandles`		Get handles of inventory items that have been used/dispensed.

📋 IAVIHistoryInProc READ/WRITE

Purpose: Clinical/medical history access, lab results, history posting. **Critical for SOAP notes.**

ProgID: `AVIHistoryInProc`

METHOD	PARAMETERS	ACCESS	DESCRIPTION
`GetFirstByDate`	`DateString`	READ	Find first history entry by date.
`GetNextByDate`		READ	Next history entry by date.
`GetLastByDate`		READ	Last history entry by date.
`GetNextByDateKey`		READ	Key-based date navigation.
`GetFirstByCode`		READ	Find history by code.
`GetLastByCode`		READ	Last entry by code.
`GetPrevByCode`		READ	Previous entry by code.
`HistoryType`		READ	Get type of current history entry.
`GetMoreStuffEx`		READ	Extended field retrieval.
`GetMoreStuffAndNormals`	`FieldMins, FieldMaxs`	READ	Get values with normal ranges (lab results).
`GetTestHandles`		READ	Get lab test handles.
`GetAnalyteHandles`	`TestHandle`	READ	Get individual analyte handles for a test panel.
`PostHistory`		WRITE	Post/commit a history entry. ⚡
`AddHistory`	`Quantity`	WRITE	Add a new history line item. ⚡
`RefillHistory`	`EnteredBy`	WRITE	Record a prescription refill. ⚡

📝 IAVIEntryInProc READ/WRITE

Purpose: System table entry management — add/read codes and descriptions.

ProgID: `AVIEntryInProc`

METHOD	PARAMETERS	ACCESS	DESCRIPTION
`GetTableHandle`	`TableCode`	READ	Get handle for a system table.
`GetEntryHandles`		READ	Get all entry handles in current table.
`GetEntriesByTable`	`TableName`	READ	Get entries by table name.
`GetFieldValuesByName`		READ	Get field values by field name.
`EntryCode`		READ	Get current entry's code.

METHOD	PARAMETERS	ACCESS	DESCRIPTION
`AddNewEntry`	`EntryDescription`	WRITE	Add a new entry to a system table. ⚡

📊 IAVISystemTableInProc READ

Purpose: Lookup table access (species, breeds, colors, allergies, etc.).

ProgID: `AVISystemTableInProc`

Methods overlap with IAVIEntryInProc for table navigation.

IAVIOrderInProc WRITE CRITICAL

Purpose: Purchase order creation and item management. **This is the key to automated ordering.**

ProgID: `AVIOrderInProc`

METHOD	PARAMETERS	ACCESS	DESCRIPTION
`CreateOrder`	`Vendor`	WRITE	Create a new purchase order for a vendor. ⚡
`AddItem`	`ItemCode`	WRITE	Add an inventory item to the current order. ⚡

Workflow: `CreateOrder("IDEXX")` → `AddItem("CBC")` → `AddItem("CHEM17")` → order is ready for submission.

IAVIFileInProc READ/WRITE CRITICAL

Purpose: Direct V2$ file access with authentication. **This is the "god mode" interface** — raw access to any field in any table.

ProgID: `AVIFileInProc`

METHOD	PARAMETERS	ACCESS	DESCRIPTION
`Login`	`Password`	AUTH	Authenticate before file operations. 🔐
`OpenFiles`	`Path`	READ	Open V2$ files at specified path (e.g., "C:\AVImark").
`CloseFiles`		READ	Close opened files.
`GetFileHandles`		READ	List all available file handles.
`GetFileNames`		READ	List all file names (CLIENT.V2$, ANIMAL.V2$, etc.).
`GetFileHandle`	`FileName`	READ	Get handle for specific file.
`GetRecordCount`	`FileHandle`	READ	Count records in a file.
`GetFieldNames`	`Param2`	READ	Get field names for a file's record structure.
`GetFieldValues`		READ	Get field values for current record.
`SetFieldValues`		WRITE	Modify field values by position. ⚡
`SetFieldValuesByName`		WRITE	Modify field values by name. ⚡
`AddRecordEx`		WRITE	Add a new record to a file. ⚡

⚠ **Warning:** This interface provides unrestricted write access to the underlying data files. Incorrect use could corrupt the AVImark database. Always test on a clone/backup first.

IAVIEstimateInProc READ

Purpose: Estimate/quote entry access.

ProgID: `AVIEstimateInProc`

METHOD	PARAMETERS	DESCRIPTION
`GetEstimateEntryHandles`	`EstimateHandle`	Get line items for an estimate.

IAVIAppointmentInProc READ

Purpose: Appointment scheduling, doctor/room calendars, block-offs.

ProgID: `AVIAppointmentInProc`

METHOD	PARAMETERS	DESCRIPTION
`GetAppointmentHandlesByDate`	`StartDate, EndDate`	Get appointments in date range.
`GetAppointmentsByDoctor`	`DoctorID, StartDateTime, IncludeInactiveDeleted`	Get appointments for a specific doctor.

METHOD	PARAMETERS	DESCRIPTION
`GetAppointmentsByRoom`	`RoomCode`	Get appointments for a specific room.
`GetBlockOffExsByDoctor`	`DoctorID`	Get extended block-off times for doctor.
`GetBlockOffExsByRoom`	`RoomCode`	Get extended block-off times for room.
`GetBlockOffsByDoctor`	`DoctorID`	Get block-off times for doctor.
`GetBlockOffsByRoom`	`RoomCode`	Get block-off times for room.

IAVICensusInProc READ

Purpose: Whiteboard/census — in-clinic patient tracking, diagnosis access.

ProgID: `AVICensusInProc`

METHOD	PARAMETERS	DESCRIPTION
`Diagnosis`		Access diagnosis information for census patients.

5. API Method Catalog — Complete Summary

INTERFACE	METHOD	ACCESS	PARAMETERS
IAVIClientInProc	`GetFirstByName`	R	NameKey → Value
	`GetNextByName`	R	NameKey
	`GetFirstByNameKey`	R	NameKey
	`GetNextByNameKey`	R	
	`GetFirstByPhone`	R	PhoneKey
	`GetNextByPhone`	R	
	`GetFirstByPhoneKey`	R	PhoneKey
	`GetFirstByWholePhoneKey`	R	PhoneKey
	`GetNextByPhoneKey`	R	
	`GetNextByWholePhoneKey`	R	
	`GetClientsByEmailOrPhone`	R	Email, Phone
	`GetPatientHandles`	R	ClientHandle, IncludeInactive
	`GetAccountingHandles`	R	
	`GetByFolder`	R	Folder
	`GetFirstByMoreStuff`	R	MoreStuffKey
	`GetNextByMoreStuff`	R	
	`GetFirstByMoreStuffKey`	R	MoreStuffKey
	`GetNextByMoreStuffKey`	R	
	`GetMoreStuff`	R	RecordHandle → FieldNames, FieldValues
	`CreateInvoice`	W	Date
IAVIPatientInProc	`GetByCode`	R	Code
	`GetByTagNo`	R	TagNo, Microchip
	`GetReminderHandles`	R	PatientHandle
	`GetHistoryHandles`	R	
	`GetAppointmentHandles`	R	
	`GetViewHandles`	R	ViewName
	`PrintPatientChart`	R	PatientRecord
	`GetModelHandles`	R	TreatmentHandle
	`Values`	R	
IAVITreatmentInProc	`GetModelHandles`	R	TreatmentHandle
IAVIInventoryInProc	`GetInventoryUsedHandles`	R	
IAVIHistoryInProc	`GetFirstByDate`	R	DateString
	`GetNextByDate`	R	
	`GetLastByDate`	R	
	`GetNextByDateKey`	R	
	`GetFirstByCode`	R	
	`GetLastByCode`	R	
	`GetPrevByCode`	R	
	`HistoryType`	R	
	`GetMoreStuffEx`	R	
	`GetMoreStuffAndNormals`	R	FieldMins, FieldMaxs
	`GetTestHandles`	R	
	`GetAnalyteHandles`	R	TestHandle

INTERFACE	METHOD	ACCESS	PARAMETERS
	`PostHistory`	W	
	`AddHistory`	W	Quantity
	`RefillHistory`	W	EnteredBy
IAVIEntryInProc	`GetTableHandle`	R	TableCode
	`GetEntryHandles`	R	
	`GetEntriesByTable`	R	TableName
	`GetFieldValuesByName`	R	
	`EntryCode`	R	
	`AddNewEntry`	W	EntryDescription
IAVIOrderInProc	`CreateOrder`	W	Vendor
	`AddItem`	W	ItemCode
IAVIFileInProc	`Login`	AUTH	Password
	`OpenFiles`	R	Path
	`CloseFiles`	R	
	`GetFileHandles`	R	
	`GetFileNames`	R	
	`GetFileHandle`	R	FileName
	`GetRecordCount`	R	FileHandle
	`GetFieldNames`	R	Param2
	`GetFieldValues`	R	
	`SetFieldValues`	W	
	`SetFieldValuesByName`	W	
	`AddRecordEx`	W	
IAVIEstimateInProc	`GetEstimateEntryHandles`	R	EstimateHandle
IAVIAppointmentInProc	`GetAppointmentHandlesByDate`	R	StartDate, EndDate
	`GetAppointmentsByDoctor`	R	DoctorID, StartDateTime, IncludeInactiveDeleted
	`GetAppointmentsByRoom`	R	RoomCode
	`GetBlockOffExsByDoctor`	R	DoctorID
	`GetBlockOffExsByRoom`	R	RoomCode
	`GetBlockOffsByDoctor`	R	DoctorID
	`GetBlockOffsByRoom`	R	RoomCode
IAVICensusInProc	`Diagnosis`	R	

6. Security Analysis

Authentication

- `IAVIFileInProc.Login(Password)` requires authentication before raw file access
- Password is likely the server password from `AVImarkServer.ini` (hex-encoded hash)
- Other interfaces may or may not require prior Login — needs testing

Access Control

- COM server is 32-bit InprocServer32 — runs in the calling process's context
- No apparent role-based access control within the COM API
- Once authenticated via Login(), appears to have full access to all operations
- The server process (AVImarkServer.exe) runs as LocalSystem — high privilege

Risk Assessment

RISK	SEVERITY	MITIGATION
Write operations could corrupt data	HIGH	Always test on clone server first. Backup before write operations.
No transaction/rollback mechanism visible	HIGH	Verify each write with subsequent read. Implement application-level logging.
Password may be extractable from INI	MEDIUM	Restrict file access to AVImark directory. Use OS-level permissions.
COM server could crash AVImark if misused	MEDIUM	In-process COM runs in caller's space, not server's. But shared files could deadlock.

7. Integration Opportunities

With Current ETL Pipeline (30 tables, 2.58M rows in Supabase)

INTEGRATION	INTERFACES NEEDED	COMPLEXITY	VALUE
Client search API	IAVIClientInProc	Low	Real-time client lookup from web app
Appointment viewer	IAVIAppointmentInProc	Low	Live schedule with doctor/room filtering
Vaccine reminder automation	IAVIPatientInProc	Medium	Auto-generate recall lists with GetReminderHandles
Purchase order automation	IAVIOrderInProc	Medium	CreateOrder + AddItem from Supabase analytics
SOAP note integration	IAVIHistoryInProc	Medium	Read/write medical history, post AI-generated notes
Price update tool	IAVIFileInProc	High	Bulk fee schedule updates via SetFieldValuesByName
Full bidirectional sync	IAVIFileInProc	High	Read via GetFieldValues, write-back from web app

eOrdering Integration (SOAP Web Services)

AVImark includes XML transformation templates for electronic ordering:

- `AVP_eOrderingAdminMailAccounts.xtr` — Mail configuration
- `AVP_eOrderingCatalogItemsAlternates.xtr` — Catalog alternates
- `AVP_eOrderingCateglogItems.xtr` — Catalog items (number, description, manufacturer, price, stock_quantity)
- `AVP_eOrderingContactLogin.xtr` — Distributor login credentials
- `AVP_eOrderingShippingAddress.xtr` — Shipping addresses

These use SOAP/XML and could be leveraged for distributor-integrated automated ordering.

8. Automation Use Cases

🛒 Automated Inventory Ordering

```
1. Supabase analytics detects low inventory (PO history + usage patterns)
2. System generates suggested order list
3. Staff approves (or auto-approve for routine items)
4. COM: CreateOrder("IDEXX") → AddItem("CBC") → AddItem("CHEM17") → ...
5. Order appears in AVImark's PO system
6. Distributor receives order via eOrdering SOAP integration
```

📋 AI SOAP Notes

```
1. Vet completes visit (services billed, prescriptions written)
2. AI reads visit context via GetHistoryHandles + GetMoreStuffEx
3. AI generates SOAP note from structured data + voice notes
4. Vet approves/edits
5. COM: AddHistory() → PostHistory() writes note back to AVImark
```

📅 Smart Scheduling

```
1. Client requests appointment via web portal
2. GetAppointmentsByDoctor checks availability
3. GetBlockOffExsByDoctor excludes blocked times
4. System suggests optimal slots
5. COM: (appointment creation method - needs further investigation)
```

💰 Automated Price Updates

```
1. Annual CPI/inflation analysis from Supabase (already built — 126 items updated in 2026)
2. Generate proposed price changes
3. Manager approves batch
4. COM: Login() → OpenFiles("C:\AVImark") → SetFieldValuesByName on PRICE.V2$ records
```

9. Registration Status & Next Steps

Current Status

> **Registration: FAILED (Silent)**
> `regsvr32 "C:\AVImark\AVImarkCOMServer.dll"` completes without error code but does not write ProgIDs or CLSIDs to the Windows registry. The 32-bit variant (`C:\Windows\SysWOW64\regsvr32.exe`) hangs with a dialog box (inaccessible on headless server).

Possible Causes

1. **Runtime dependency:** DLL may require AVImark client application context (not just the server service)
2. **Missing Delphi runtime packages:** Large Delphi COM servers sometimes need BPL packages
3. **Server vs. Workstation:** May need to be registered from an AVImark workstation, not the server
4. **License check:** DllRegisterServer may verify AVImark license before registering
5. **Path dependency:** May need to run regsvr32 from C:\AVImark directory

Next Steps (Priority Order)

#	ACTION	RISK	EXPECTED OUTCOME
1	Try `regsvr32` from C:\AVImark directory	Low	May resolve path dependency
2	Try registration from an AVImark workstation	Low	Client context may satisfy runtime check
3	Use `tlbimp` or OleView to extract full TLB	None	Complete method signatures with data types
4	Try direct COM activation via CLSID from TLB	Low	May work without registry if CLSID is known
5	Analyze DllRegisterServer with debugger	Low	Identify exact failure point
6	Contact Covetrus for COM API documentation	None	Official docs may exist for integration partners
7	Try Python win32com with manual CLSID	Low	Bypass ProgID registration entirely

10. Appendix

A. Related Files on ROS-SVR

FILE	SIZE	PURPOSE
`AVImarkCOMServer.dll`	74.7 MB	COM server (this analysis)
`AVImarkServer.exe`	—	AVImark server process (PID 12048, running)
`AVImarkServer.ini`	200 B	Server config (password hash, port 9191)
`STSeries.tlb`	38 KB	ST Series type library (charting/graphing component)
`IVLS\IDEXXClient.ocx`	—	IDEXX VetLab Station ActiveX control (registered)
`AVP_eOrdering*.xtr`	1.8–8.8 KB	Electronic ordering SOAP/XML templates (5 files)

B. Server Environment

Hostname	ROS-SVR
OS	Windows Server 2019 (10.0.17763)
Architecture	x64
AVImark Version	2022.10.6
AVImark Port	TCP 9191
AVImark Path	C:\AVImark
Data Files	~100+ V2$ files, 1.08 GB main DB
ETL Pipeline	30 tables, 2.58M rows → Supabase (nightly 2:00 AM)